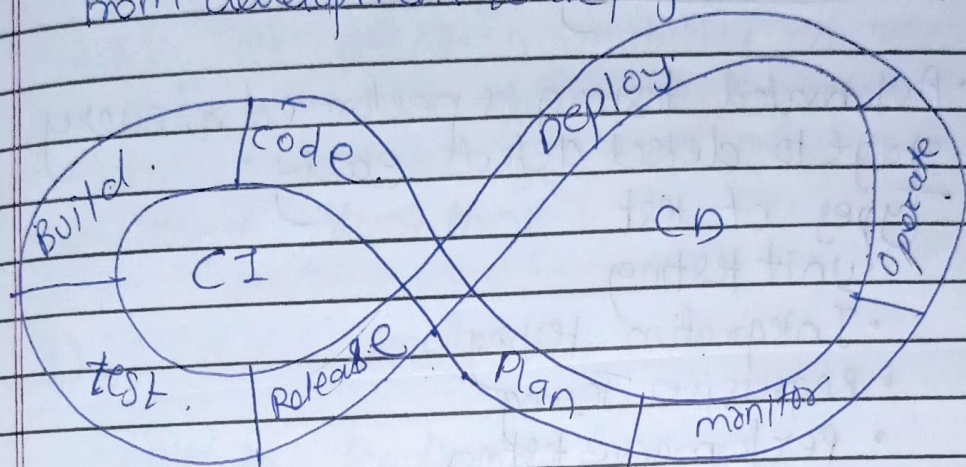


Continuous Integration & Test Driven Development.

~ Pratik Patil

Q1 Explain CI/CD pipeline:

- CI/CD
- A devops pipeline is a sequence of automated steps through which software changes from development to deployment.



1) Continuous Development ~~(CD)~~

- Involves planning the project defining requirements & writing the code.
- project scope, features, goals & timelines are decided.
- Developer writes code using programming language.
- Version controls like git, github are used to track & manage code changes.
- Code is broken into small, manageable commits to make integration & testing easier.

2) Continuous Integration (CI)

- 1) merges codes developed by multiple developers

into a single repository

- Ensures that code works well with existing system & does not break functionality.

3) Continuous Testing

- Automated testing is performed at every stage to detect defects early.

• Types of test

- unit testing
- Integration testing
- Regression Testing
- Performance testing

- Test results are automatically sent to the development team

4) Continuous Deployment (CD)

- Automatically Deploying to production without manual approval
- This is done by predefined tests. Once the tests are passed the system pushes the updates directly to software users.

5) Continuous monitoring

- Application are monitored in real time for performance, security & availability checks.
- Detects issues like high CPU usage, slow response time.

- Issues are directly sent to devops team.

6) Continuous operation

- Focus on automatic release & updates to reduce downtime.
- Ensures 24/7 availability of application.
- Enables - blue green deployment or rolling updates so that updates happens without downtime.

7) Continuous feedback

- Collects feedback from
 - monitoring tools
 - user analytics
 - Direct customer input

- Feedback is analyzed to identify bugs & system satisfaction.
- Helps in improvement

Q] Define Continuous Integration (CI). Explain the benefits to Continuous Integration

Continuous Integration is a developer's practice where developers frequently merge their updated code into a shared source repository, and each merge is automatically validated through automated builds and tests. It avoids long integration delays, detects bugs early, & ensures that the main branch always remains in a stable working condition.

Benefits of CI

- 1) Fast feedback
- 2) Better Collaboration → Work in team rather than isolation
- 3) Faster delivery → Automated build & test reduces waiting time & delay
- 4) Higher Quality → prevent bugs & software reliability is improved
- 5) More efficiency → Team spends less time in QA & testing

Benefits of CD

- 1) Quick release to users → New features can be deployed whenever available
- 2) Low release risk → updates are smaller & incremental, reducing failures
- 3) Improved product quality → Automated validation ensures stable builds reach production.
- 4) Efficient infrastructure → Reusable automated env. reduce setup efforts.
- 5) Better team transparency → Pipeline progress & results are visible across teams

Q] Difference between ~~CI & CD~~ Continuous delivery & deployment

Continuous Delivery	Continuous Deployment
1) Code is delivered to testing automatically	1) Code is delivered directly to production automatically
2) Manual approval needed for production	2) No manual approval
3) Human checks before going live	3) Only failed tests stop the release.
4) Release schedule is controlled by team	4) Release happens as soon as code passes tests.
5) Suitable when business wants control over releases	5) Suitable when organization wants fast releases.
6) Requires story telling but human decision.	6) Requires very strong automated testing.
7) Developers may release manually	7) Dev focus on coding not release.
8) Release freq: frequent but controlled	8) Release frequency: very high.
9)	

Q] What is continuous Integration and does it have any impact on quality of product & Time?

★ Impact of CI on Time to Market, ★

1) Fast detection of issues

- Since every commit is tested automatically bugs are found immediately instead of at the end of a project.

2) No waiting for large integration

- Developers don't work separately for long periods, integration happens daily or multiple times a day.

3) Less re-testing time

- Code is validated continuously, teams don't need a long testing cycle before release.

4) Avoids complex merges

- frequent merging prevents big merge conflicts that block development.

5) Automated pipeline saves time

- Build + test automation replaces manual process by automation

6) Supports rapid releases cycles

- CI removes unnecessary dependencies improves release cycle

7) Small incremental updates

- Teams releases features in short bursts instead of waiting for full releases.

★ Impact of CI on product Quality

1) consistent code quality

- code passes through ^{automated} validation keeping code clean & stable

2) prevent defective release builds

- CI checks code at every step, reducing faulty features being shipped to user

3) Reduced bug accumulation

- Bugs are not stored to fixed later, they are ~~store~~ solved as they appear.

4) Better collaboration

- Teams communicate more because they work on shared repo.

5) Higher reliability

- Code is tested on an Integration Machine. Broken builds are identified immediately

6) Early bug fixing reduces cost

- Fixing small issues early is cheaper than repairing major defects later
- CI ensures these mistakes are caught during development

7) Improved Customer Satisfaction

- Fewer issues better experience.

Q1 Explain different deployment strategies.

1) Recreate Deployment

- The older version of the app is completely shut down ~~and~~ before releasing or deploying new one.
- This results in downtime during updates.
- This is usually performed in or during off-hours usually at night.

2) Ramped (Rolling update)

- New version is released gradually by replacing instance one by one.
- Replacing 1 server → wait → replace next.
- Works well but may cause temporary mixed-version issues.

3) Blue/Green Deployment

1) Two identical environment.

1) Blue → current environment

2) Green → New version, tested & ready

- After testing in Blue, you can switch all traffic from ~~Green~~ blue to green.
- If there's problem instantly switch blue.
- But this is costly.

4) Canary deployment

- The new version is released to a small percentage of users (10%).
- Rest of users will still use old version.
- Performance & feedback are ~~constantly~~ closely monitored.
- If all set release the model gradually.

Q7] Explain role of continuous integration servers.
What operations a CI server can execute?
→ Roles of CI server

1) Automates the CI process.
→ The CI server manages source code repositories & automatically runs build & test tasks whenever developers commit new code.

2) Provides feedback to developers
→ When a build fails or a test fails, the CI server immediately notifies developers, reducing delay & improving collaboration.

3) Reduce manual work
→ It removes the need to manually compile, test or deploy code, helping teams work faster & more consistently.

Operations a CI server can execute.

1) Fetch & pull source code.
• It retrieves latest committed code from the repository.
• Ensures that the server always uses the most recent version.

2) Build/Compile the application
• Converts source code into a deployable artifact.
• Detects compile-time errors & dependency issues.

3) Execute automated tests.

- Runs unit tests or tests suites to verify correctness of the merged code.
- Ensures the stability before moving forward.

4) Generate build artifacts

- Stores successful builds as artifacts for later deployment.
- Keeps a clean history of stable releases.

5) Send notifications

- CI server informs developers about build results through alerts or dashboards.
- Enables fast resolution of errors.

Q8] What are build stage? Write step wise process to generate builds

→ The build stage in CI/CD pipeline step where the CI server uses the source code, checks code syntax, dependencies errors. It ensures that the code is correct, runnable, & deployable stage before testing or deployment.

Step-wise process to generate Builds

Step 1 → Select CI/Build server.

- Choose tools like Github, Azure pipeline, Jenkins.
- The build server will run all automated build tasks.

Page No.
 Date

Step 2 → Fetch Latest Source Code

- CI server pulls the latest code from the version control.
- Ensures the build uses most recent commit.

Step 3 → Set Up Build Environment

- Install required libraries, modules & dependencies.

Step 4 → Compile the Source code

- CI server compiles the code to check for compilation errors & missing dependencies.

Step 5 → Analysis & Code check

- Build Pipeline helps in finding quality issues early.

Step 6 → Create Build artifacts

- A successful build generates deployable artifacts such as
 - JAR/WAR files
 - Executables
 - Docker images
 - Packages
- These artifacts are stored for testing & deployment.

Step 7 → Store & publish build

- Build artifacts are stored in artifact repository.
- The output is now ready for testing or deployment.

Q7 Why do we need continuous database integration in DevOps? List the tools used for CI in DevOps.

→ 1) Database changes must match application changes when code changes frequently. The database schema should also update continuously to avoid mismatches.

2) Reduce integration issues. Continuous syncing of DB schema avoids last-minute failure like missing tables, wrong columns or invalid data types.

3) Fast & safe deployment. Automated DB updates help faster releases without manual SQL execution, reducing errors.

4) Prevents data loss or corruption. CI allows applying small incremental DB updates, avoiding fatal issues.

5) Consistency across environments.

- Development, staging & production db remains consistent when schema & scripts are version-controlled.

6) Helps faster delivery to market. Smooth DB & application integration supports rapid feature rollout in DevOps pipelines.

Tools used for CI in DevOps

1) Jenkins

a) Bamboo

b) CircleCI

c) Github actions

d) Gitlab CI

Q3) What is use of code repository server? How to select a good repository server?

Ans: Use of code repository server

1) Central storage of source code

- A code repository server stores the entire project's source code at one common location so that developers can easily access, update & manage it.

2) Supports version control

- It helps in tracking changes & maintain history of every modification.

3) Enables team collaboration

- Multiple developers can work on the same project without conflicts.

4) Provides secure backup of code

- It protects the organization's code by storing it safely & preventing code loss due to system failure.

5) Supports automation in CI/CD pipeline

- Build server automatically pull code from repo server to start CI/CD pipeline.

6) Manages project documentation

- Repo server also holds supporting file like scripts, configuration files, documentation etc.

* How to select good code Repo server

1) Supports Version Control System (VCS)

- The repo server must fully support VCS & should integrate smoothly with commit commands & branching.

2) Provides strong security

- Must offer secure access, user permissions, encryption & private repositories to protect sensitive code.

3) Easy integration with CI/CD pipeline tools

- Should integrate with CI servers like Jenkins, Github Actions, Bamboo etc. for automated builds.

4) Good stability & performance

- Repo should handle large code files, multiple projects without slowing system.

- 5) Support for collaboration features.
 - Should offer pull request, code reviews, issue tracking & team communication support.

Q) Outline imp of code repo in development & delivery.

- 1) Centralized storage of code
- 2) Enabled Collaboration Among developers
- 3) Support Version Control & History Tracking
- 4) Facilitates continue Integration (CI)
- 5) Improves Code Quality & Stability
- 6) Enables Faster & reliable Delivery.
- 7) Provides Backup & Security.

Q) How to create Build reports? Explain the process of putting a build in a shared location & releasing it.

→ # How to create Build Reports

1) Enable Reporting in CI Tool:

CI servers like Jenkins, Azure pipelines generate reports by enabling reporting plugins or modules

2) Collect Build information

- Report includes build number, build time, commit ID, developer name & test results.

3) Capture logs & Test Results

- Build logs, test outputs & warnings/errors are automatically recorded as part of the report.

4) Generate Summary Report

- The CI tool compiles all details into a readable HTML, XML or PDF report.

5) Publish the Report

- Reports are saved to a specific directory or published on CI dashboards.

Process of putting a Build in shared location & releasing it.

Step 1 → Create successful Build

Step 2 → Store Build in shared repo.

Step 3 → Tag the Build version.

Step 4 → Notify Team.

Step 5 → Deploy to Test.

Step 6 → Validate the Build

Step 7 → Release to production.

1) Explain what technology changes are required in the development tool chain to enable DevOps.

1) Adoption of version control system.

- Teams must use distributed repositories like Git.
- Helps track every change, enables collaboration.

2) Use of Automated Build Tools.

- Manual building must be replaced with tools like maven, Gradle, Ant, Maven.
- Ensures consistent, repeatable, & fast builds for every code change.

3) Integration of CI server.

- Tools like Jenkins, Bamboo automate code integration, testing & reporting.
- Provides immediate feedback & reduces integration issues.

4) Automated Testing frameworks.

- Reduce manual testing efforts.

5) Configuration management Tools.

- Tools like Chef, Puppet manages server configuration automatically.

6) Containerization & Virtualization.

- Tools like Docker & Kubernetes package applications with all dependencies.

7) Deployment Automation Tools.

- CI/CD pipelines uses Azure DevOps, Jenkins, Github Actions.
- Automate releases, reduces errors & speeds up delivery.

8) Monitoring & logging Tools.

- Tools like Grafana help track performance, errors & usage.
- Necessary for continuous feedback & improving reliability.

QJ → Explain Integration testing

- Integration testing is a software testing process where two or more modules are combined & tested together to check how they interact with each other.
- It ensures that data flow, communication, & dependency between modules work correctly after combining them.
- The goal of integration testing is to detect interface defects, such as mismatched data formats, incorrect API calls, module-to-module failures.
- It is mostly done after unit testing & before system testing ensuring the combine units behave as expected.
- Integration testing is very useful when module depends on external services, such as login systems, payments gateways, database connectors or third-party APIs.
- It helps developers identify issues early when modules are combined, reducing overall debugging time.

eg → Login + Dashboard System

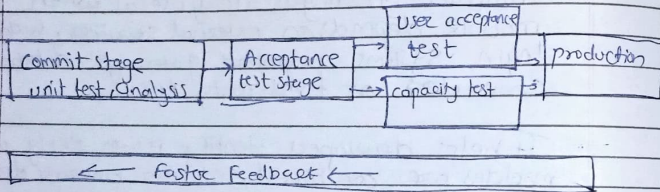
- Suppose module A = Login Module
- Module B = Dashboard module

unit testing checks both individually
But in integration testing, you test:

- 1) When the user enters correct credentials in login module.
 - 2) The system should correctly pass under ID/data to dashboard module.
 - 3) Dashboard should load correct user profile/options.
- If anything not worked as expected.
integration testing checks the connection between these two modules.

Q] Draw & explain basic deployment pipeline.

→ Env. becoming more like production-environment →



1) Commit stage

- pipeline starts when a developer commits code into the repository.
- CI system triggers automatic tasks like:
 - code compilation

- unit testing
- code analysis
- Creating instances.

purpose → ensure the code is stable before going further.

2) Acceptance Test Stage

- CI server runs large set of functional integration tests.
- This is time consuming automated test stage.
- Tests are executed in parallel so results come quickly (usually within 1-2 hours)
- If test pass pipeline moves further.

3) User acceptance test (UAT)

- It is realistic env. where system behaves like production.
- Users or stakeholders validates whether new features worked as expected.
- Ensures product meets customer needs & Business rules.

4) Capacity Test

- Checks the system performance under expected load.
- Ensures app can handle real-world traffic before going live.
- Helps avoid performance issues in production.

5) Production stage

- final step of pipeline.
- Build is deployed into the live env.

- No manual intervention if Continuous deployment is used.
- User start using the new version of the app

Q) Comments on Trade-offs in the deployment pipeline.

→ A) Monolithic Deployment - trade-offs

1) Single pipeline for entire applications

2) High risk of downtime → If anyone change fail, entire app go down.

3) Large blast radius → failure in one affects all because everything is tightly coupled

4) Slower release cycle

- Adding new features takes longer because the entire app must pass deployment checks
- Leads to infrequent releases.

5) Difficult rollback.

- If deployment fails, undoing the changes is complex because all components are deployed together

B) Microservice Deployment.

1) Each service has its own deployment pipeline.
• Every microservices is deployed individually, allowing isolated updates.

2) Reduced risk of downtime

- If a single services fails, only that services

becomes unavailable.

- The rest of the system still works, reducing impact.

3) Faster & more frequent releases

- Because each service is smaller & deploys independently, releases are more freq & less risky

4) Better scalability & fault isolation

- only the required services are scaled or restarted, not the whole application.

5) Higher operational complexity.

- many small services mean many pipelines, monitoring systems, and deployments to manage.

Q) Write short note on commit stage of deployment pipelines.

- The commit stage is the first & most imp. stage of the deployment pipeline.
- It starts immediately when a developer commits code to the version control system.

Key points

- 1) Triggered automatically on each commit:
 - whenever developer pushes code to the repository, CI server starts a new pipeline instance.

2) Verify code correctness early
 - The main purpose is to detect errors as early as possible before they move further in pipeline

3) Includes essential build activities

3) Provides fast feedback
 - Developers get immediate alerts if the commit breaks the build

4) Acts as a quality gate
 - Only when the commit stage succeeds, the pipeline automatically moves to the acceptance test stage

5) Ensures stable, deployable builds
 - prevents faulty code from moving further,

#. Main task performed in commit stage

1) Code Compilation - The new code is compiled to ensure there are no syntax errors.

2) Unit Testing - Small test cases are executed to check the correctness of units or modules

3) Code Analysis - Automated tools check coding standards, code smells & bugs

4) Create Installers - If everything is successful, the system generates binary files or build packages

Q) Explain the steps involved in managing Defect Backlog

1) Do Nothing

- If no stakeholder is concerned about certain defects, no action is taken
- This applies when defects have no impact on functionality or business value.

2) Filter out the Noise

- Remove low-priority, irrelevant or noisy defects that do not need action.
- Helps in cleaning the list and focusing only on meaningful defects.

3) Stop Logging Low-Impact Defects

- Logging too many small defects increases noise & takes unnecessary efforts
- A way to control backlog size is to avoid very minor, insignificant bugs
- Only meaningful & impactful bugs should be added to the backlog

4) Close off Low Impact defects

- Another method is to close old, low-impact defects instead of fixing them.
- Useful when fixing minor bugs brings no business value.
- Risk is that the same error may arise by users again

5) Prune the Backlog

- Eventually review & clean the backlog list.
- Items that become irrelevant due to system updates, new versions or design changes can be removed.

6) Batch the Defects

- Group related defects and handle them together as a batch.
- Even if individual defects are low priority the whole batch may become significant.
- Helps product owners prioritize defect groups instead of single issues.

7) Blitz the Defects

- After major releases, allocate a full sprint dedicated only to fixing defects.
- This helps to handle a large no. of bugs quickly.

8) Fix Some defects every sprint

- In every sprint, dedicate a fixed portion of team time to repair defects.

9) Automate the Test

- Use automated testing tools to identify & resolve many defects quickly.

87

Write short note on Automated acceptance Test Best practices

1) Automate tests to reduce human errors.

2) Perform tests in parallel.

- CI divides Acceptance testis into smaller suites & run them simultaneously, reducing total execution time.

3) Ensure tests verify customer requirements.

4) Maintain a stable environment.

- For reliable test results, acceptance tests run in an environment similar to production & that failure reflect real issues.

5) Automated acceptance should confirm that new changes do not break old functionalities in app.

6) After commit stage is successful, the acceptance test stage must start without manual intervention, ensuring continuous workflow.

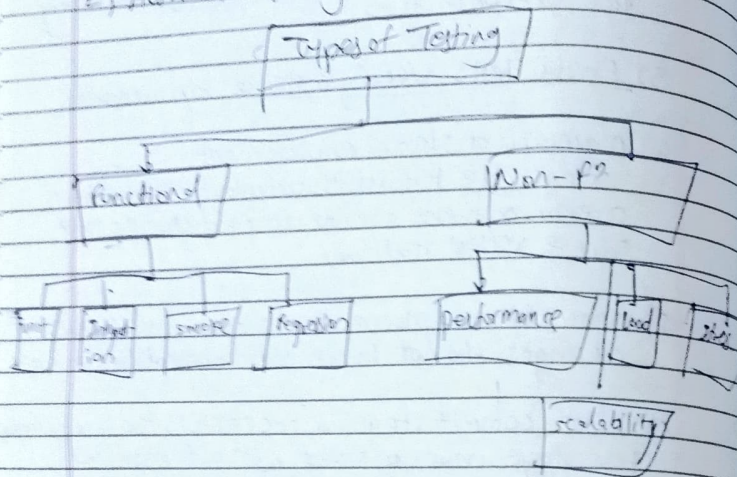
7) Acceptance testing provides valuable insights that help identify defects earlier & prevent faulty builds from moving forward.

8) Explain diff types of testing and how to implement them in Continuous deployment.

→ There are majorly two types of Testings

1) Manual Testing

2) Automated Testing.



A) Fⁿ testing

• Fⁿ testing is a process of software testing performed to ensure Fⁿ requirements & specification are satisfied by the software. It is easy to perform & Fⁿ testing helps to enhance behavior of the system.

1) Unit testing

- Tests individual units/modules of code.
- Ensures each unit behaves as expected.
- eg → Testing the app of `add(x,y) fn`.

2) Integration testing

- Tests how multiple units/modules work together.
- Ensures interaction betⁿ components or third-party codes behaves correctly.

3) Smoke testing

- Quick health check after every build.
- Ensures major functionalities work before deeper testing stages.

4) Regression Testing

- Ensures new changes don't break existing functionality.
- performed after fixes, updates or ~~improvements~~ improvements.

5) Non-Functional Testing

1) Performance Testing -

- measures speed, responsiveness & stability of the software.
- checks CPU usage, memory usage, data transfer.

2) Load Testing

- Tests behavior under expected user load.
- eg → login page tested with lower vs 500 users.

3) Stress Testing

- pushes system beyond normal limits.
- E-commerce website during festival sale.
- Helps to identify breakpoint before system failure.

4) Scalability Testing

- checks how well the system scales up or down.
- Ensures resources can adjust dynamically to load change.

5) Security Testing

- Ensures system defences against unauthorised access & vulnerabilities.

How to implement Testing Strategy in CI

1) Automate All possible Tests.

- functional & non-functional tests run automatically after each commit.
- Continuous Testing tools automate correctness tests, integration tests, smoke tests & acceptance tests.

2) Execute Tests in a pipeline structure

- Unit test → Integration test → Acceptance test → performance tests.
- Tests run parallel to reduce time.

3) Test in multiple Environment

- Run tests in production like environment.

4) Use Infrastructure as Code

- Automated provisioning of test env. reduces errors and ensures consistency.

5) Prioritize Test Feedback

- Fail fast → dev. gets immediate feedback on issues.

6) Continuous monitoring & Reporting

- collect test result, performance data, logs continuously.
- Helps identify failures early & maintain stability.

Diff between Integration Testing & Unit Testing	
Point	Integration Testing
1) Testing focus	Tests individual units/modules of code separately
2) Objective	To ensure each unit behaves as expected
3) Who performs it?	Developers
4) Scope	Very small scope
5) Tools used	PyTest, JUnit, NUnit
6) Defects found	Logic errors inside a unit

Diff between Integration testing & Conventional Testing	
Point	Conventional Testing
1) When perform	late in SDLC
2) Approach	Big Bang & extend
3) Error Detection	Errors detect late
4) After cost	More as late error detection
5) Automation	Manual testing
6) Role	Does not support CI/CD

Unit Testing	
Point	Unit Testing
1) Testing focus	Tests how two or more module work together
2) Objective	To ensure interaction bet ⁿ units is correct
3) Who performs it?	Dev & testers
4) Scope	Large Scope
5) Tools used	Postman, Selenium, etc testing tools
6) Defects found	Interface mismatches, data flow issues, integration failure

Integration Testing	
Point	Integration Testing
1) When perform	Continuously
2) Approach	Incremental testing
3) Error Detection	Early
4) After cost	Less as defects error early
5) Automation	Automated
6) Role	used in CI/CD

Explain diff betⁿ CI/CD

Continuous Integration	Continuous Delivery
1) Developers frequently merge code to shared repo	1) Code is automatically delivered or deployed after passing test
2) Focus is on integrating code changes early	2) Focus is on delivering releases quickly & reliably
3) Detects errors early during integration	3) Ensures code is always ready to release
4) Involves steps like compile, unit test, static analysis	4) Involves steps like acceptance testing, capacity testing, staging & production
5) Goal is to ensure new code works with existing code	5) Goal is to push working code to user faster
6) Mostly developer-oriented	6) Mostly operation release-oriented
7) Outputs a tested build	7) Outputs a deployable or deployed build
8) Runs frequently - many times per day	8) Runs after PR is merged, may be daily/weekly or fully automatic
9) Tools -> Git, Jenkins, GitHub Actions	9) Tools -> Jenkins CD, AWS Code Deploy

unit 6 DevOps Tooling & Case Studies

Page No.	
Date	

Q1) What is Version Control? Explain its type and benefits.

→ Version control systems are software tools that help software teams manage changes to source code over time. It is the practice of tracking & managing changes to software code.

- Version control is the tool used to track changes in code and project files, maintain history, & allow collaboration among team members.

- It helps developers work simultaneously without overwriting each other's work & enables restoring older versions when required.

Maintaining VC involves following practices

- 1) Uses VCS: Store all project files in VCS tools like Git, etc.

- 2) Create Regular commits → Commit changes frequently with proper updation message to maintain history

- 3) Use Branching Strategy → Create branch for new feature, bug fixing or experimentation

- 4) Merge changes systematically: - After testing, merge feature branches into the main branch using Pull request & merge requests.

5) Maintain backup & central Repository

6) Resolve conflicts carefully - when multiple devs edit the same file merge conflicts must be resolved properly.

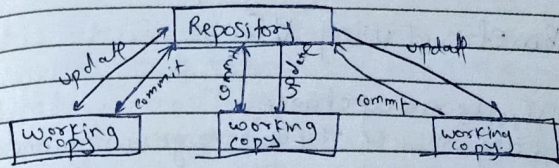
Types of Version Control System

1) Local Version Control Systems:

- Stores all version of files locally on the developer's system
- Simple to use but no collaboration support.
- Risk of data loss because everything is stored in one place.

2) Centralized Version Control Systems (CVCS)

- A centralized version control system (VCS) is VCS where all users are working under some central repository.
- This central repo. can be located at server or on dev's local machine.
- CVCS is used where team of dev's needs to share code on their changes.
- Two things required to make your changes visible to others which are:
 - You commit
 - They update.



• If central server fails, all work will stop

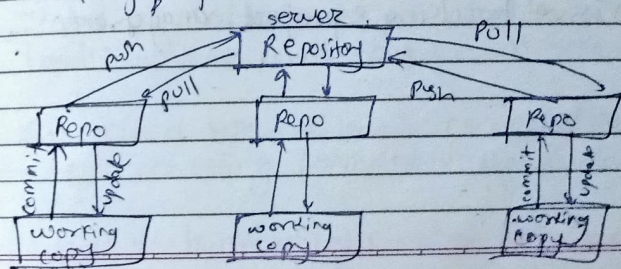
3) Distributed VCS

• Distributed VCS systems contain multiple repositories. Each user has their own repository and working copy.

• DVCS brings a local copy of the ~~change~~ complete repo to every team member's computer. So they can commit, branch, & merge locally.

• To make changes visible to other 4 things are required.

- You commit
- You push
- They pull
- They update.



Benefits of using VC

- 1) Tracks every change
- 2) Easy rollback → If anything goes wrong, undo.
- 3) Improves collaboration
- 4) Supports branching
- 5) Reduce errors.
- 6) Provides backup & recovery
- 7) Improves productivity.

Q] How Github is used in Version Control?

- 1) Central Remote Repository
- 2) Collaboration betⁿ multiple developers
- 3) Branching & merging
- 4) Pull Request for Code Review
- 5) Tracks every change automatically
- 6) Backup & safety.
- 7) Integration with devOps tool
- 8) Issue tracking & project management.

Q]

What is serverless orchestration in Kubernetes?

• serverless orchestration means managing application workflow without managing servers yourself. Kubernetes handles the infrastructure, scaling & execution automatically.

• It is used to run multi-step tasks or workflows. eg → sequences of functions, steps or services that run one after another.

• Developers only write the logic of the steps, not the infrastructure. Kubernetes take care of running, stopping, retrying and scaling.

• AWS step fⁿ is a common ex of serverless orchestration, where workflows of AWS lambda fⁿ are managed automatically.

• It handles branching logic, retries and error handling without dev. writing complex code.

• It improves app reliability, because failed steps can be retried or skipped based on rules.

• It reduces efforts as we don't have to handle server, containers or VM resources.

• Diff cloud services can be connected easily for ex, connecting lambda, storage, db, etc.

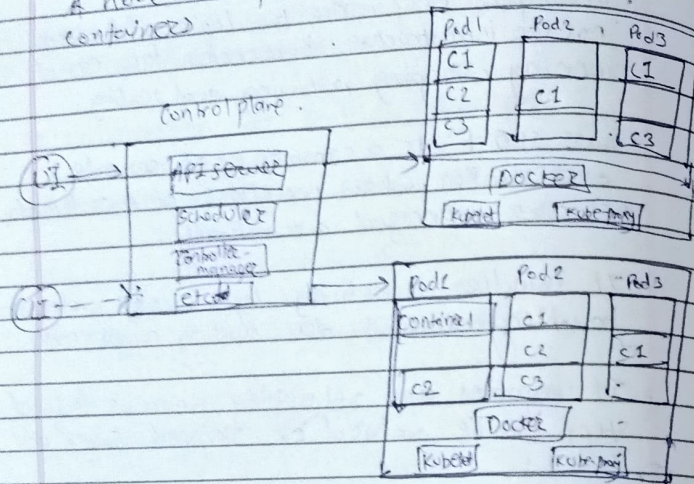
• Focus is on business logic not on infrastructure.

Q7] Kubernetes architecture.

• Kubernetes cluster has two main components: the control plane & worker nodes. The control plane is used as compute resources.

• The control plane hosts the components used to manage the Kubernetes cluster.

• Worker nodes can be VM or physical machines. A node hosts pods, which run one or more containers.



The main components of Control Plane are-

- **API server** - the entry point for all cluster commands.
- **Scheduler** - decides on which node a pod should run.
- **Controller-manager** - manages diff controller like replication, node controller etc.
- **etcd** - key-value store that keeps cluster configuration and state.

- Each worker node contains
 - **Kubelet** - agent that communicates with the control plane.
 - **Kube-proxy** - manages networking between pods.
 - **Container Runtime** - runs the containers.

• App in Kubernetes run inside Pods, which are the smallest deployable units containing one or more containers.

• The control plane constantly checks actual state vs desired state. If any thing goes wrong, Kubernetes will automatically restarts, reschedules or replaces the workload.

• Kubernetes architecture ensures scalability, fault tolerance, and automatic management of containerized applications.

- Q) Explain notes on Docker:
- Docker is a software platform used to build, package & run applications inside containers.
 - A container includes application code + all dependencies so the app runs the same on any system.
 - Docker helps devs deploy apps faster and avoid problems like "works on my device".
 - It allows easy movement of applications between development, testing & production environments.
 - Docker ensures lightweight, isolated & consistent environments.

* Components of Docker

1) Image →

- Images are like blueprints for containers.
- They store the application code, libraries, & instructions for building containers.
- You can get image from Docker hub or create our own images by including specific instructions within a file called Dockerfile.

2) Container →

- Containers are live instance of image on which app or its independent module runs.
- In app image is class and container is instance of that class. This allows operational efficiency.

- They start quickly & use fewer resources compared to VM.

3) @Registry

- A registry is a repository that stores images.
- The Default registry is Docker Hub.
- Developers can upload (push) & download (pull) images easily.

4) Docker Engines

- Docker Engine is the core runtime that runs containers.
- It follows a client-server architecture with three parts:
 - Docker Daemon → runs on host machine, manages images & containers.
 - Docker Client (CLI) → used to give Docker commands.
 - Rest API → enables communication betⁿ client & daemon.

Benefits of Docker →

- 1) Portability → App run same everywhere.
- 2) Consistency → No conflict betⁿ libraries or system settings.
- 3) Fast deployment → Containers start within sec. make S/C faster.
- 4) Resource → Container uses fewer resources compare to VM.
- 5) Easy scalability → App can be scaled up/down quickly based on load.
- 6) Better Collaboration → Devs can share containers through Docker images.
- 7) Version control for environments → Everying version can be stored and reused.

- 8) App Isolation $\rightarrow$ One container's failure does not affect others
- 9) Wide Ecosystem $\rightarrow$ many tools & ready-to-use img support modern development workflows

Q] Write a note on Jenkins and its role in CI/CD

- Jenkins
- Jenkins is the open source automation server used for building, testing & deploying software.
 - It helps developers automate repetitive tasks in the SDLC.
 - Jenkins supports plugins, which extend its features and allow integration with many DevOps tools.
 - It provides web-based dashboard where jobs are created, configured & monitored.
 - Jenkins automate the entire process from writing code $\rightarrow$ test $\rightarrow$ deploy $\rightarrow$ releasing.

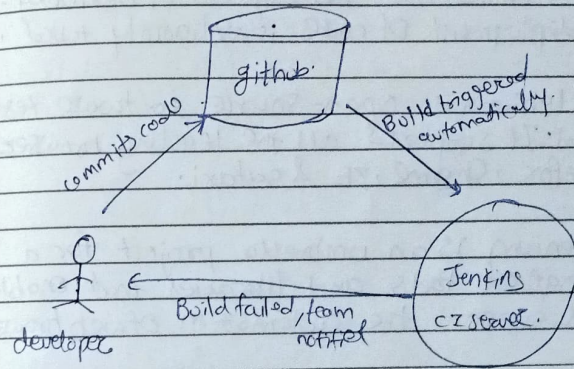
Roles of Jenkins in CI.

- Jenkins automatically pulls the latest code from version control (git/github).
- Every time a developer commits code, Jenkins build the project automatically.
- It runs two unit tests & integration tests to detect bugs early.
- If the build fails, Jenkins immediately notify the developer.
- It ensures the main codebase always stays clean, stable & bug free.
- This reduces integration problems & saves time spent on manual checking.

- Jenkins allows developer to integrate code frequently enabling faster development cycles.

Roles of Jenkins in CD.

- After CI is successful, Jenkins can automatically package & prepare the app for deployment.
- It can deploy application to different environment like Testing, Staging or production.
- Jenkins integrates with tools like Docker, AWS, Kubernetes for automated pipeline deployment.
- It ensures that App are delivered quickly, safely & repeatedly.
- In CD, Jenkins even pushes the app to production automatically without human ~~at~~ intervention.
- It reduces manual work & ensures smooth & reliable releases.



Q7 Explain Continuous testing in Selenium.

→ Continuous testing is the process of testing the software or application continuously on each step throughout the project lifecycle which provides the quality of software at every step of CD process by testing early.

- The older way of testing was very time consuming where along with the dev team the QA team was also needed to evaluate the quality, & thus this process was time consuming.

- The organization want faster delivery & deployment so they need better testing software which automates the process.

- Hence The continuous testing was introduced in which the development, delivery, testing & deployment of code is continuously tested.

- Selenium is open-source software testing tool. It supports all the leading browsers like Firefox, Chrome, IE & Safari.

- Selenium is an umbrella project for a range of tools and libraries that enable & support the automation of web browsers.

- Selenium scripts can be reused across different browsers without changing code.

- Continuous testing pipelines (eg Jenkins, Selenium) automatically run regression tests whenever new code is pushed.

- This helps detect issues early, improve software quality & speeds up the release cycle.

- Because Selenium is open source it has strong community support & wide ecosystem of tools.

- Q1) Write a note on Prometheus
- Prometheus is an open-source monitoring & alerting tool mainly used in cloud and DevOps environments.
- 1) It collects metrics from applications, containers, servers & services to help understand system performance.
 - 2) Prometheus stores data in a time-series database, meaning data is saved with timestamp, so changes can be studied over time.
 - 3) It uses a pull-based model, where Prometheus regularly pulls ^(samples) metrics from targets through HTTP endpoints.
 - 4) Prometheus uses a special query language called PromQL, which helps developers analyze metrics & create dashboards or alerts.
 - 5) It works very well with Kubernetes, where it monitors pods, nodes, CPU, memory & cluster health.
 - 6) Prometheus provides a built-in alerting system, where alerts are sent to an Alertmanager, which can be notified via email, Slack, etc.
 - 7) It is highly scalable, lightweight, & efficient, which makes it suitable for modern microservices-based systems.
 - 8) Prometheus is widely used to detect failures early, track performance issues, ensure reliability of applications.
 - 9) Since it is open-source, it has a strong community & integrates easily with tools like Grafana for graphical dashboard.

- Q2) What is bug tracking? Comment on tools used for bug tracking.
- Bug tracking is the process of identifying, recording, monitoring & managing defects (bugs) found in a software system.
- 1) It helps development & testing teams keep track of issues from the time they are reported until they are fixed and closed.
 - 2) Bug tracking ensures no defect is forgotten, and each issue is handled based on priority & severity.
 - 3) It improves communication between ^{developers} ~~developers~~, testers & project managers.
 - 4) It helps in analyzing repeated issues, understanding software quality, & improving the overall development process.
 - 5) A good bug tracking system provides reports, notification, dashboards & history of all bugs activities.
 - 6) Bug tracking tools also integrate with CI/CD pipelines and version control to support devops workflow.

Tools used for Bug Tracking.

1) JIRA:

- One of the most widely used bug and project tracking tool.
- Allows teams to create bug reports, assign them, add comments, set priorities, track progress.
- Supports agile methodologies like Scrum & Kanban.

- 2) Bugzilla - Open-source bug tracking tool by Mozilla.
- Provides advanced search, reporting & workflow customization.
 - Suitable for large teams needing detailed bug management.

- 3) Redmine - Open-source project management & bug tracking tool.
- Supports issue tracking, Gantt charts, time tracking & collaboration.

- 4) MantisBT - Simple & user-friendly bug tracking system.
- Allows email notifications, issue categorization & plugin support.

- 5) Trello - Uses boards, lists & cards to manage bugs visually.
- Most useful for small teams who want lightweight tracking.

- 6) GitHub Issues - Integrated with Version Control System.
- Developers can create, assign & link bugs directly to code commits.
 - Useful in DevOps and Open-source projects.

Q7) Explain various VC tools? Compare them & how are they helpful?

					Bitbucket
Features	Git	SUN	Mercurial	Perforce	GitLab/GitLab
1) Type	Distributed vs	Centralized	Distributed	Centralized	Cloud based
2) Speed	Very fast	Slow	Fast	Fast	Fast (depends on Git)
3) Branching	Easy	Complex	Easier than SVN	Good but heavy	Very easy using UI
4) Repository	Local + remote	Only central	Local + remote	Central	Remote cloud repo
5) Usage	Most popular	Legacy sys.	Research Project	Large enterprise	Teams collab began Git
6) Learning	Medium	Easy	Easy	Med	Easy.
7) Best for	Open-source Devs, CI/CD	Simple projects	Python program	Gaming, enterprise-scale	Collaboration CI/CD integration

Types of VC

- 1) Git
- 2) SUN
- 3) Mercurial
- 4) Perforce
- 5) GitHub
- 6) GitLab
- 7) Bitbucket

How they are helpful

- 1) Track changes in code
- 2) Easy to restore previous versions
- 3) Support Team Collaboration
- 4) Avoids Code Conflicts.
- 5) Better project management.
- 6) Improved Code Quality.
- 7) Enables Continuous Integration.

Q) Illustrate the use of containers while creating container based app design. Explain various containers used in relation with DevOps

→ (In first part write the benefits of the dockers.) Turn 5 pages back & see on right page

Various Containers used in relation with DevOps

→ 1) Docker Containers.

2) Kubernetes (Containers (Pods))

- Kubernetes runs containers by grouping them inside "Pods"
- Used for orchestration, scaling, load-balancing & self-healing of app.

3) Linux Containers

- Lightweight or sevel virtualization
- Offers system-level containers.

4) Containerd / CRZ-O Containers.

- Lightweight container runtimes used by Kubernetes for managing containers efficiently.
- focus on simplicity, security, & Kubernetes compatibility

5) serverless Containers

- Used in AWS Lambda, Google Cloud Functions.
- Run short-lived fⁿ inside containers without developers managing any servers.

Q) Tools used for continuous testing

→ 1) Selenium

2) Jenkins

3) Kubernetes / Docker

4) GitHub Actions

5) JUnit

6) Postman

7) Apache Jmeter

Q) which tools are mostly used in devops? write note about one

→ write note on Jenkins

Tools

1) Git

2) Jenkins

3) Docker

4) Kubernetes

5) Grafana

6) Prometheus

7) Selenium

8) Ansible